

6 Mise à Jour Immutable du State

6.1 Pourquoi l'Immutabilité ?

React compare l'ancien et le nouveau state pour décider de re-render. Si on modifie directement le state, React ne détecte pas le changement.

△ Important

Règle d'or : Ne jamais modifier state directement

```
// INTERDIT - Mutation directe
const [user, setUser] = useState({ name: 'Ahmed', age: 20 });
user.age = 21; // Ne marche pas !

// CORRECT - Créer nouveau objet
setUser({ ...user, age: 21 });
```

6.2 Tableaux : Méthodes Immutables

Listing 24 – Opérations tableaux immutables

```
const [items, setItems] = useState(['a', 'b', 'c']);

// AJOUTER élément
setItems([...items, 'd']); // ['a', 'b', 'c', 'd']

// SUPPRIMER élément (par index)
setItems(items.filter((item, index) => index !== 1)); // ['a', 'c']

// MODIFIER élément
setItems(items.map((item, index) =>
  index === 1 ? 'NEW' : item
)); // ['a', 'NEW', 'c']

// REMPLACER tout
setItems(['x', 'y', 'z']);
```

★ Astuce

Méthodes à éviter (mutent tableau) :

- push(), pop(), shift(), unshift()
- splice(), reverse(), sort()

Méthodes OK (retournent nouveau tableau) :

- map(), filter(), concat(), slice()
- Spread operator [...items]

6.3 Objets : Spread Operator

Listing 25 – Mise à jour objets immutable

```
const [user, setUser] = useState({
  name: 'Ahmed',
  age: 20,
  address: {
```

```
    city: 'Agadir',  
    zip: '80000'  
  }  
});  
  
// Modifier propriété simple  
setUser({ ...user, age: 21 });  
  
// Modifier propriété nested (attention !)  
setUser({  
  ...user,  
  address: {  
    ...user.address,  
    city: 'Casablanca'  
  }  
});
```